

SORTIA 2.0 / CIÊNCIA DA COMPUTAÇÃO

Sortia 2.0: do vetor à árvore

Heapsort → AVL

AMANDA NOGUEIRA
SILVIANE SOUZA
RODRIGO NUNES
LIZ CRISTINA

UMA IDEIA EM TRÊS FORMAS

Vetor

Pai e filho em posições.

Heap

Decisões e trocas.

Árvore

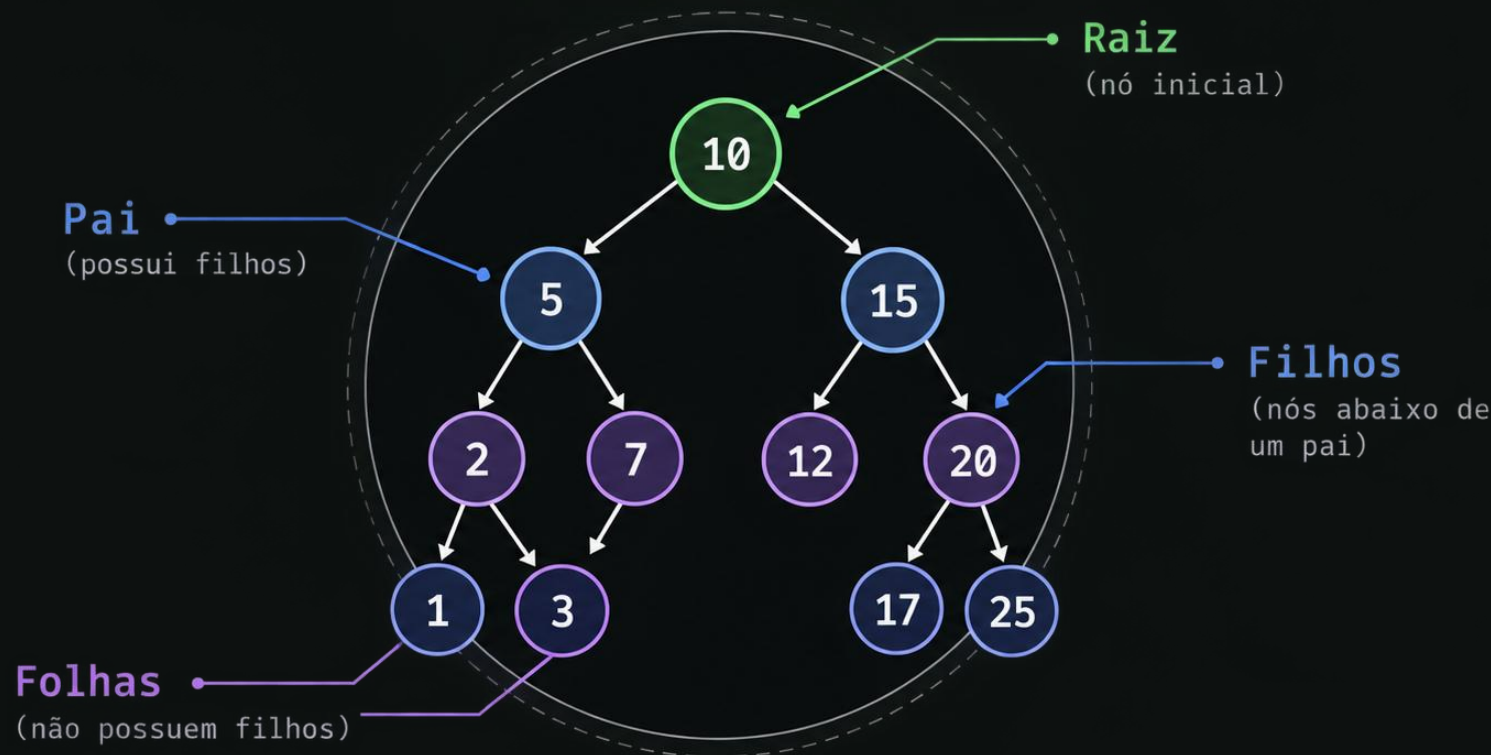
Estrutura visível.

O PONTO DE PARTIDA

0 vetor funciona, mas esconde a hierarquia

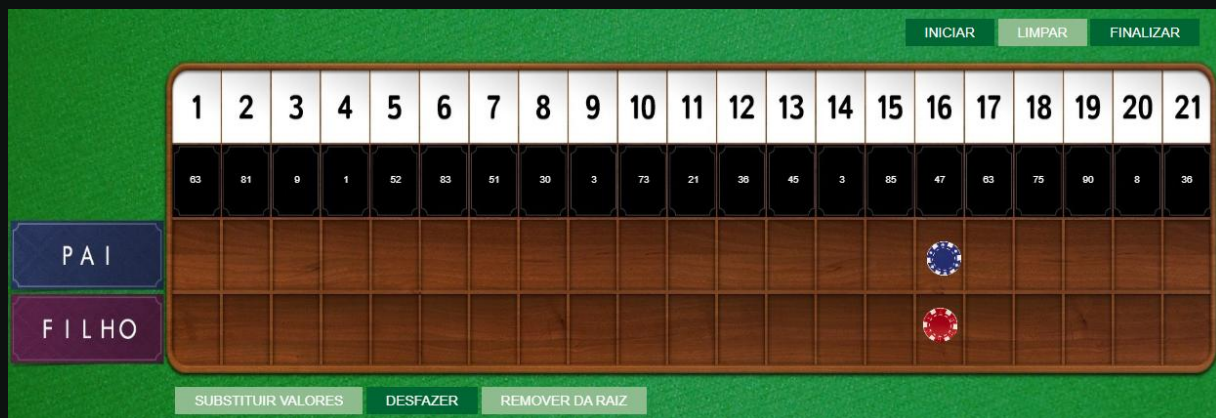
Sortia 2.0 simula o Heapsort com um vetor de 21 posições, ponteiros Pai e Filho e trocas manuais.

ÁRVORE DE BUSCA BINÁRIA



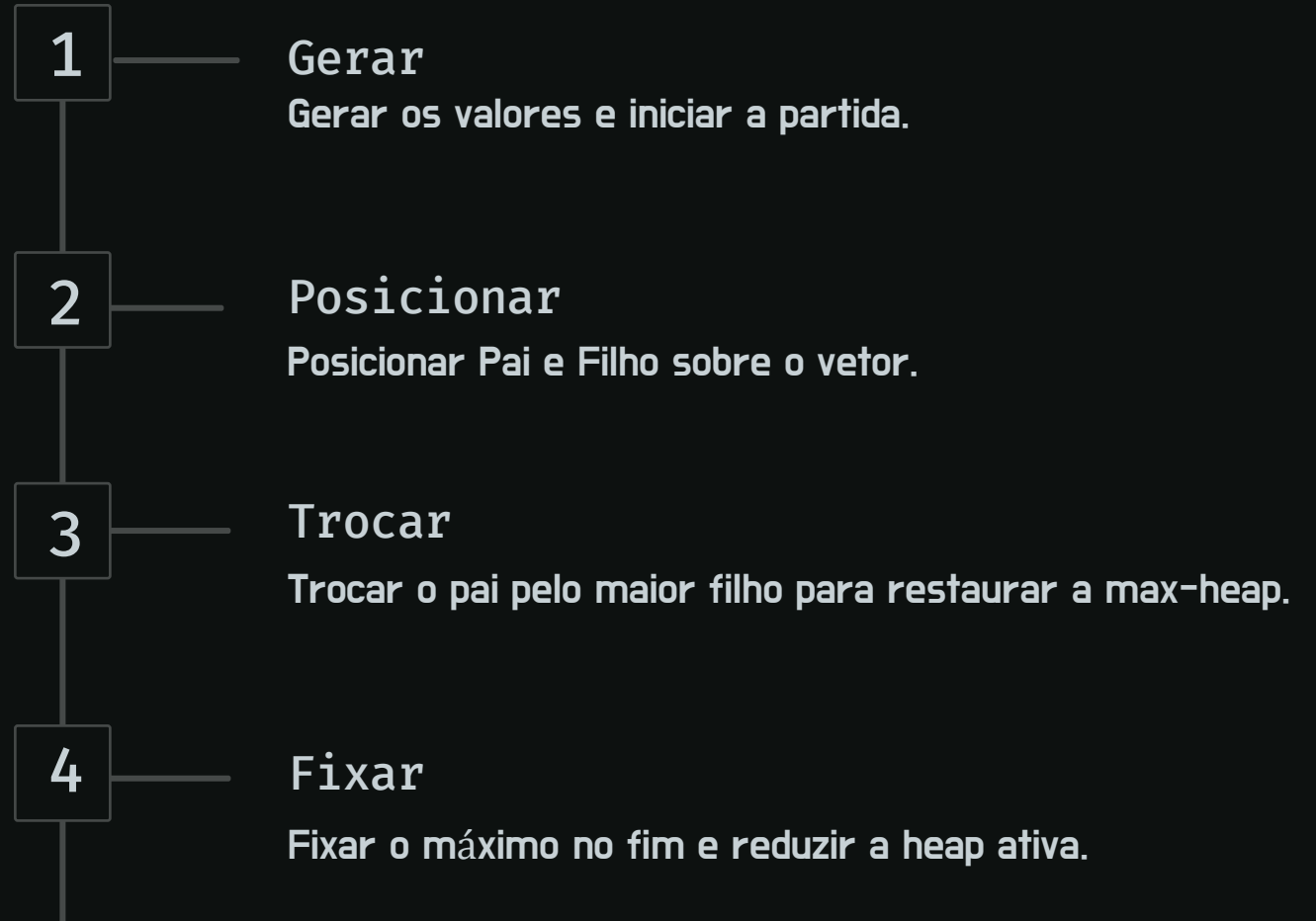
O MESMO VETOR, AGORA COMO ÁRVORE

O jogador pratica as comparações, mas ainda não enxerga a árvore que o vetor representa.



O jogo transforma a ordenação em decisões

O vetor deixa de ser apenas uma
sequência: cada troca exige
comparar, escolher e restaurar a
max-heap.



```
=====
JOGO DE HEAPSORT – APRENDER FAZENDO
=====
Em um vetor com índice i: esquerdo = 2i+1; direito = 2i+2.
Regra da max-heap: todo pai deve ser maior ou igual aos filhos.

1 – Usar números aleatórios
2 – Digitar os números
Escolha: 1

=== FASE 1 – CONSTRUIR A MAX-HEAP ===
Percorreremos os pais de baixo para cima.

Analisando o nó de índice 2...

Vetor: [0:8] [1:10] [2:40] [3:62] [4:42] [5:27] [6:63]
Heap ativa: [8, 10, 40, 62, 42, 27, 63] | Ordenados: []
Árvore lógica (índice:valor):
  0:8
  1:10  2:40
  3:62  4:42  5:27  6:63
O nó 2:40 viola a propriedade de max-heap.
Digite os índices que devem ser trocados (ex.: 1 4) ou 'dica': dica
Dica: compare os dois filhos e troque o pai com o filho de maior valor.
O nó 2:40 viola a propriedade de max-heap.
Digite os índices que devem ser trocados (ex.: 1 4) ou 'dica': █
```

A nova versão leva o mesmo raciocínio para o terminal



Entrada

Números aleatórios ou de 4 a 12 inteiros digitados.



Estrutura

A árvore lógica aparece por níveis, mas o vetor continua sendo a estrutura de jogo.



Feedback

Troca correta soma acerto; erro e dica afetam o placar.

UM EXEMPLO COMPLETO

Construir a
heap, ordenar e
pontuar: tudo
fica rastreável

150

pontos no placar final

15 ACERTOS · 0 ERROS · 0 DICAS

RASTRO

VETOR

RESULTADO

Original

[88, 59, 26, 86, 70, 33, 32]

Construir a heap

Ordenado

[26, 32, 33, 59, 70, 86, 88]

Fixar o máximo

Vetor: [0:26] [1:32] [2:33] [3:59] [4:70] [5:86] [6:88]
Heap ativa: [] | Ordenados: [26, 32, 33, 59, 70, 86, 88]

Parabéns! Ordenação concluída.

Original: [88, 59, 26, 86, 70, 33, 32]

Ordenado: [26, 32, 33, 59, 70, 86, 88]

Placar: 150 pontos (15 acertos, 0 erros, 0 dicas)

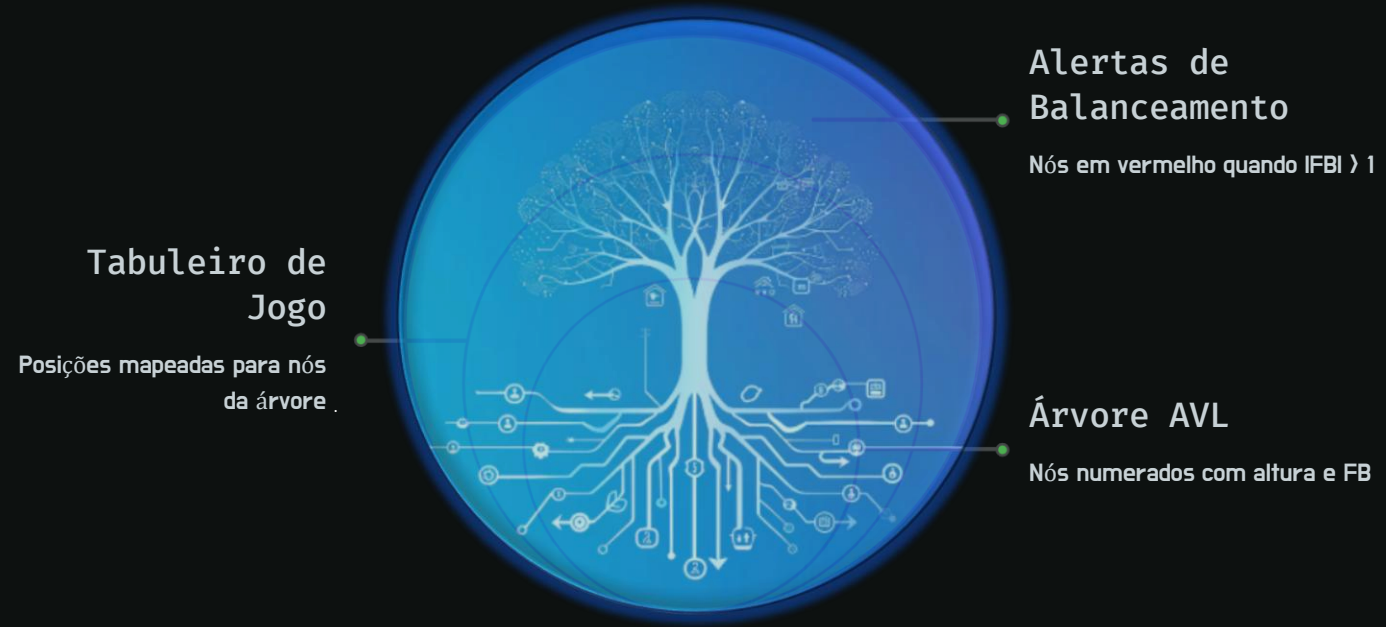
MUDANÇA PEDAGÓGICA

A atividade deixa de ser “ver o algoritmo” e passa
a ser executar, justificar e receber feedback.

PRÓXIMA EVOLUÇÃO

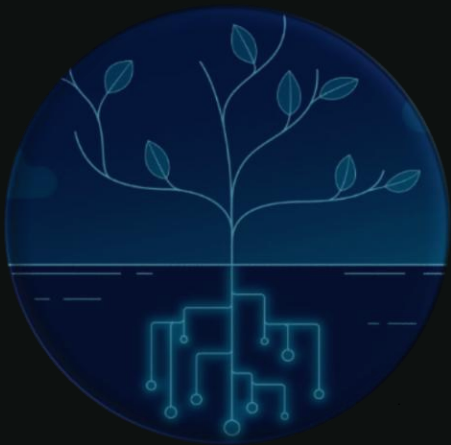
A próxima evolução é tornar a árvore visível e jogável

- Upgrade proposto: preservar a aprendizagem ativa e transformar o jogador em responsável por uma árvore AVL balanceada
- Cada número vira uma chave: o jogador insere respeitando esquerda menor e direita maior
- Altura e fator de balanceamento ficam visíveis em cada nó; o estado válido é $FB \in \{-1, 0, +1\}$



O jogador vence quando mantém o equilíbrio

Inserir a chave, recalcular alturas, identificar o caso, escolher a rotação e atualizar a árvore.



Vitória

Todas as chaves inseridas e a AVL válida.

Derrota

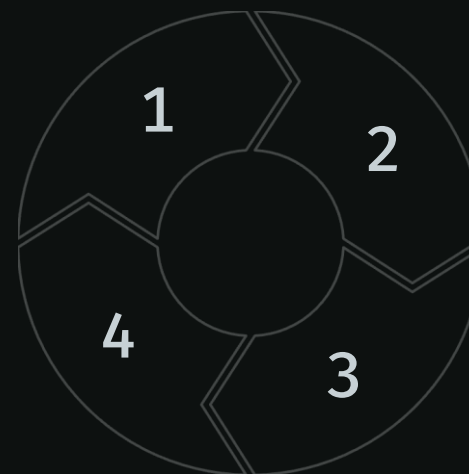
Erros críticos deixam a árvore alta, desbalanceada e menos eficiente.

Inserir
A próxima chave.

Atualizar
Voltar à próxima chave.

Medir FB
Alturas no caminho.

Escolher rotação
LL, RR, LR ou RL.



LL



RR



LR



RL

Do vetor visível à estrutura compreendida.